

Best practices to follow when creating or evolving your analytics tracking

Tracking plan best practices to get you successful



Analytics tracking best practices



Franciska Dethlefsen

January 10, 2021

8 min read

Contents

[Start with your goals and metrics](#)

[Don't forget event and user properties](#)

[Establish consistency and keep it simple](#)

[Determine where to capture events](#)

[Keep separate dev and production environments](#)

[Enforce your tracking plan](#)

[Assign an owner](#)

[Document everything](#)

In a previous blog post we explored the [why, how and what of creating a tracking plan](#). If you're just getting started with analytics tracking at your company, we suggest you start there. In this blog post we're going to focus on some of the best practices we've seen work well for companies.

To quickly recap, a tracking plan is a living document (or it can live in a tool like [Iteratively](#)) and it usually outlines what events and properties to track, what they mean and where they are tracked. It helps codify a single source of truth for your analytics and provides your developers with the details they need to instrument the analytics tracking (or schema) in your product's code base.

And why do you need one? Well, without one it's very hard to know what events you're capturing in your product and what they mean. It also helps ensure the data you capture is consistent across your sources (think iOS, Android, web, backend) and gives data consumers an understanding of what data they are able to explore for analysis in tools like Amplitude and Mixpanel, or directly in your data warehouse.

Besides the tracking plan you also need a process around how you define, instrument and verify your analytics tracking. This process usually involves your product managers, data analysts and developers.

In this post we'll explore some ways to ensure you and your team are successful and can reap the benefits of a tracking plan and process and take your analytics to the next level 🚀.

Start with your goals and metrics

It's crucial that you start by outlining your metrics and then work your way down to what events you need to properly report on that metric. Without a link between your goals, metrics and events you'll most likely end up with a sprawling tracking plan and data you don't actually need, while missing out on events crucial to your business.

Goal	Increase acquisition by 15% in Q1
Metric	Conversion Rate = User Signed Up / Unique Visitors
Event	User Signed Up
Properties	user_id, campaign, experiment, referrer, etc...

It also helps you prioritize events for instrumentation and hopefully forces product managers and data analysts to think about not only the goal or the metric of

Don't forget event and user properties

Properties are where you can capture all the details associated with an event or user. They describe the context around the event or the user and allows your analysts to be able to group, filter, and cohort.

There are two kinds of properties: event specific (e.g. the revenue amount associated with a purchase event) and user specific (e.g. demographic information about a user). Most events and users will have multiple properties associated with them and as with your events, we recommend you only capture what you need and start small.

Establish consistency and keep it simple

A major culprit of data quality issues is inconsistent naming conventions. You might have one team capturing an event as Song Played, while another team is capturing the same event as Song_Played. This leaves analysts with lots of data munging or worse yet, incomplete reports.

Agree on a naming convention for your events and properties and ensure it's clear to everyone involved with defining and instrumenting analytics tracking (or use a tool like Iteratively to enforce it easily 😊).

	Taxonomy	Example
Event naming convention	Title Case	e.g. Song Played
Property naming convention	snake_case	e.g. song_title

Along with your naming conventions settle on a framework for your events, e.g. Object-Action: first choose your objects (e.g. Song) then define actions users perform on that object (e.g. Played, Paused) to construct events like "Song Played" or "Song Paused". And finally agree on a tense (e.g. Song Played or Song Play).

Determine where to capture events

You have options when it comes to analytics tracking and it's important to understand the pros and cons to determine the optimal mix that fits your business and analytics needs. Many companies limit themselves by only capturing events client-side and not taking advantage of server-side event capture.

Collecting events on the server is more reliable and we recommend you always

Client-side tracking allows for much richer information and is where you should capture events where you do require the context for how an event occurred (e.g. for a first page view you want to capture UTM parameters and referrers to understand where the visit came from). But bear in mind that ad blockers and browser restrictions such as ITP and ETP can limit your client-side tracking and so you want to find an optimal mix of richness and reliability.

Keep separate dev and production environments

This one is straight forward, but we still see companies sending data from their development environments to their analytics destinations. Don't dirty your production data and make sure you keep your environments separate.

How you ensure that varies by analytics tool, e.g. Segment [recommends you set up different sources by environment](#) (e.g. local/dev/prod) and in Iteratively you use separate access tokens which [can be configured in Destinations](#).

Enforce your tracking plan

Many teams treat analytics tracking as an afterthought and do not apply the same practices as they would to other code. This naturally results in analytics bugs that you'll have to fix downstream, or worse, that you don't discover at all. We see many teams losing trust in their data that way and once the trust is gone it's hard to build back!

To mitigate that it's crucial you validate and enforce your analytics tracking. We've written [a comprehensive guide outlining various ways of validating your data according to its specifications](#).

To summarize, there are several ways to enforce your tracking and they usually fall into one of two categories: reactive or proactive approaches, and you can enforce your tracking plan, or analytics schema, in the client, in the pipeline and at the destination (usually a data warehouse or analytics destination like Mixpanel). We always recommend dealing with quality issues at the source, i.e. ensuring that your instrumentation matches what is spec'ed out in the first place and then verifying that with [unit testing](#) and as a part of CI/CD.

Assign an owner

Having a clear owner of your tracking plan is crucial. Accountability is needed to ensure your tracking plan is kept up to date. In another blog post, [Who should really own your tracking plan?](#), we dive into who could be that owner and how you build a process around your analytics tracking.

Event tracking is going out with every new product release. This means defining a clear process for your event tracking and entrusting the product team to take ownership by empowering them with the right tools and training.

Document everything

We can't stress the importance of up-to-date documentation. Without it, analytics tracking will easily become messy, teams will start forgetting to include tracking as a part of the release process and you start the downward spiral of not trusting your data.

Manual documentation can be tedious and easy to forget, but we strongly recommend to document the following at a minimum:

- **Analytics tracking guidelines** - An overview of the complete process, including your event taxonomy and framework, how to define new events, who's responsible for what and links to related resources.
- **Tracking plan** - The actual list of events and properties, including descriptions, from where they're tracked, since when and who's the owner. [Segment](#) and [Data-led Academy](#) offer decent templates for you to get started, but over time it becomes harder and harder to keep up-to-date.
- **Instrumentation process** - We recommend including a process document on how (to ensure) new events are implemented, down to the Jira ticket-level, requirements around instrumentation, testing, validation and more.

Many companies use Google Sheets, Notion or Confluence pages to manage these documents. With Iteratively, that's all done for you automatically, and we even sync your tracking plan to your analytics destinations such as Mixpanel, Amplitude and dbt, ensuring the whole company is in sync around analytics ✨.

Get to best practice with Iteratively

Iteratively helps data teams, product managers and engineers define, instrument, verify and collaborate on analytics tracking. We proactively solve the data quality issues that arise from inconsistent event naming and missing tracking and provide a workflow for managing the evolution of your tracking.

We ensure teams get high quality data that's ready to use by empowering them to get analytics tracking right the first time. If you're interested in trying out Iteratively for your company, [create an account today](#) or [book a demo with our team](#) to learn more.

SIGN UP FOR PRODUCT UPDATES & RELATED CONTENT

Work email

Sign up

TRUSTED BY GREAT TEAMS LIKE

ARTIFACT
UPRISING

box

THREDUP

dribbble

octopus

PRODUCT

Features

Pricing

Integrations

Security

LEARN

Documentation

Blog

COMMUNITY

Customers

GitHub

COMPANY

About us

Careers

Contact us

Legal